TRABALHO 1

Este primeiro trabalho visa desenvolver a habilidade de manipular cadeias de caracteres e arquivos-texto.

Basicamente, o trabalho consiste em processar um arquivo-texto contendo um conjunto de instruções para a criação de outros arquivos.

A ENTRADA DE DADOS

O programa processará um arquivo-texto contendo os seguintes comandos, como descrito na tabela abaixo. Cada comando pode ter um parâmetro. O arquivo-texto contém exatamente um comando por linha. O comando está separado de seu parâmetro por, exatamente, um espaço em branco. Os parâmetros podem ser:

- **string**: uma sequência de caracteres delimitado por aspas, não as incluindo. Exemplo: "abacaxi".
- **path**: é uma sequência de nomes de diretórios (pastas), separados por "/". Um path pode ser ou não terminado por um "/". Por exemplo, "/a/b/c" e "/a/b/c/" são exatamente o mesmo path.
- **texto**: todos os caracteres, incluindo brancos, até o final da linha
- **#v:** significa: o conteúdo da variável v (v pode ser x, y ou z)

comando	parâmetros	descrição
x	`string`	*concatena string ao conteúdo de x*
y	`string`	*concatena string ao conteudo de y*
z	`string`	*concatena string ao conteúdo de z*
lx		*limpa o conteúdo de x*
ly		*idem para y*
lz		*idem para z*
d	`(path | #x | #y | #z)`	*Define o diretório corrente*
a	`(string | #x | #y | #z)`	*Define o nome do arquivo*
e	`(string | #x | #y | #z)`	*Define a extensão do nome do arquivo*
c		*Cria o arquivo com nome extensão definidos pelos comandos a e e, dentro do diretório definido pelo comando d*
o		*Semelhante ao comando c, mas abre o arquivo para inclusão (append). Se o arquivo não existir, primeiro cria.*
w	`(texto | #x | #y | #z)`	*Grava uma linha no aquivo correntemente aberto. Caso não exista nenhum arquivo aberto, deve escrever na saída padrão uma mensagem de erro.*
f		*fecha arquivo*

Exemplo 1:

`x "abcd"`	*conteúdo de x: abcd*
`x "efg"`	*conteúdo de x: abcdefg*
`d ./saida/`	
`a "arq"`	
`e "txt"`	
`c`	*cria o arquivo ./saida/arq.txt*
`w #x`	*escreve em uma linha o conteúdo de x*
`f`	*fecha o arquivo criado por c*

exemplo01.txt

Exemplo 2:

`x "/home/eu"`	
`x "/docs/"`	
`x "d1"`	*conteúdo de x: /home/eu/docs/d1*
`y "a1"`	*conteúdo de y: a1*
`d #x`	*diretório do arquivo: /home/eu/docs/d1*
`a #y`	*nome do arquivo: a1*
`e "txt"`	*extensão do arquivo: txt*
`c`	*cria arquivo: /home/eu/docs/d1/a1.txt*
`lx`	
`x "Ola, galaxia"`	
`w #x`	
`f`	

exemplo02.txt

O Programa

O nome do programa deve ser `t1` e aceitar o parâmetro -f:

`t1 -f` *`arquivo`* `-o` *`dir`*

O parâmetro **`-f`** especifica o nome do arquivo de entrada e o parâmetro **`-o`**, especfica um valor default para o comando **`d`**.

A Implementação

Espera-se que o programa esteja bem dividido em procedimentos curtos. Por exemplo,

poderiam ser implementadas funções que:

- leia e retorne uma linha de um arquivo-texto
- dado um path, retorne o diretório. Ex. f("/a/b/c/d/e.txt"), produz "a/b/c/d".
- dado um path, retorne o nome do arquivo. Ex. f("/a/b/c/d/e.txt"), produz "e".
- dado um path, retorne a extensão do arquivo. Ex. f("/a/b/c/d/e.txt"), produz "txt".
- entre outros.

O Que Entregar

Submeter a sala Moodle o arquivo ***nome*.zip**, que deve conter um diretório *src*, com os fontes do programa e um makefile. Este makefile deve ter (pelo menos) o target `t1` que deve produzir o arquivo executável (dentro do diretório src).

Note que ***nome*** é alguma abreviatura do nome do aluno. A abreviatura só deve conter letras (por exemplo, **`jlsilva`**, poderia se referir ao aluno José Luís da Silva).

A Avaliação

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação se baseará em dois critérios: **(a)** compilação e testes do executável; **(b)** inspecção do código-fonte. O professor fará os teste em uma máquina virtual rodando o SO Linux.

A nota será proporcional ao número de testes executados com sucesso. Portanto, caso o programa não compile ou não execute corretamente, a nota poderá ser muito baixa!